
2026년 개인주제탐구발표대회 보고서

연 구 주 제 명:PyMuPDF 기반 외부 TTF 폰트 변환 및 최적화 시스템 구축

2026. 09 . 04 .

과 정 명	SW고급
소속 학교	대전 둔산중학교
성 명	두건호

대전교육정보원정보영재교육원

I. 탐구(연구)의 필요성 및 목적

1. 탐구(연구)의 필요성
2. 연구 목적

II. 이론적 배경

1. 선행 연구
2. 용어 정의

III. 탐구(연구) 방법 및 과정(절차)

1. 연구 방법
2. 연구 절차

IV. 연구 결과

V. 결론 및 제언

1. 결론
2. 제언

※참고문헌

I. 탐구(연구)의 필요성 및 목적 I. 탐구(연구)의 필요성 및 목적

I. 탐구(연구)의 필요성 및 목적

1. 탐구의 필요성 (Background & Necessity)

가. 디지털 문서 표준(PDF)의 고정된 레이아웃 구조와 가독성 한계

PDF(Portable Document Format)는 작성된 문서의 시각적 형태를 디바이스 환경과 무관하게 동일하게 유지하는 장점이 있지만, 바로 그 특징 때문에 텍스트의 동적 재배치(Reflow) 및 맞춤형 서체 변경이 매우 까다롭다는 구조적 한계를 지닌다.

나. 정보 접근성(Accessibility) 및 다양한 사용자 환경 고려의 필요성

시각장애/저시력자, 난독증 사용자, 혹은 고령층의 경우 특정 고가독성 서체(아구스 체, 나눔바른고딕 등)나 큰 글자 크기가 필수적이다.

그러나 기존 PDF 문서는 단순한 폰트 변경이 불가능하여 맞춤형 문서 환경을 제공하기 어렵다.

다. 폰트 교체 시 발생하는 '레이아웃 깨짐(Text Overflow/Overlap)' 문제 일반적인 PDF 편집 도구로 글꼴만 선택 후 변경할 경우, 글꼴마다 고유하게 가지고 있는 장평(Character Width)과 자간(Metrics)의 차이로 인해 텍스트가 정해진 범위를 벗어나거나 줄 바꿈이 어긋나는 레이아웃 붕괴 현상이 발생한다.

정량적 검증 도구 및 일괄 자동화 시스템의 문서 레이아웃이 제대로 유지되었는지 일일이 눈으로 확인하는 것은 비효율적이다.

따라서 문서 내부 구조 데이터를 분석해 폰트를 자동 최적화하고, 교체 전후의 레이아웃 변형을 픽셀 단위(Image Overlay)로 정량 측정·검증하는 기술적 구조가 절실히 요구된다.

2. 탐구목적(Research Objectives)

본탐구에서는PyMuPDF 라이브러리와 이미지 차분(Image Difference) 알고리즘을 활용하여, 기존 PDF의 구조적 한계를 극복하고 시각적 레이아웃을 보존하는 사용자 맞춤형 폰트 자동 최적화 시스템을 개발하는 것을 최종 목표로 한다. 구체적인 세부 목적은 다음과 같다.

가. PDF 메타데이터 구조 분석 및 텍스트 재배치 기술 구현

PDF 내 바운딩 박스(Bounding Box, 위치 좌표), 폰트 크기, 텍스트 스패ن(Span) 등 구조적 데이터를 해석하고, 기존 글꼴을 은폐(Redaction)한 뒤 맞춤형 폰트로 자동 교체·재구성하는 엔진을 구현한다.

나. 장평 오차 보정을 위한 동적 스케일링(Auto-Scaling) 알고리즘 설계
서체 변경에 따른 장평 차이로 텍스트가 바운딩 박스를 넘치는 레이아웃 오류를 방지하기 위해, 박스 내 텍스트 수용 여부를 감지하여 폰트 크기를 자동으로 미세 조정(-0.5pt 단계)하는 보정 로직을 개발한다.

다. 수준의 오버레이(Pixel-level Overlay) 레이아웃 검증 체계 구축
원본 PDF와 폰트 최적화 PDF를 동일 DPI 이미지로 렌더링한 후, 이미지

차분 연산을 통해 픽셀 불일치율(Mismatch Rate, %)을 정량적 지표로 계산하고 레이아웃 보존 성능을 입증한다.

라. 문서 처리 및 가독성 환경 표준안 제시

본 시스템을 통해 사용자 맞춤형 접근성을 높이는 동시에, 디자인 통일성이 필요한 대량의 기업/기관 PDF 양식을 일괄 변경·관리할 수 있는 자동화 솔루션의 가능성을 제시한다.

II. 이론적 배경

1. 선행 연구

가. PDF 구조 분석 및 동적 텍스트 재배치(Reflow) 연구

기존 디지털 문서 처리 연구에 따르면, PDF(Portable Document Format)는 PostScript 언어 기반으로 제작되어 각 문자의 절대 위치 좌표(x, y)를 고정 렌더링하는 방식을 취한다. 이로 인해 서체나 글자 크기를 변경할 때 문서 전체의 틀이 깨지는 한계가 있음이 지목되어 왔다.

이에 대한 해결책으로 PyMuPDF 라이브러리를 활용하여 PDF 내 메타데이터(Block, Line, Span)를 추출하고 텍스트 바운딩 박스(Bounding Box)를 계산하여 동적으로 서체를 재배치하는 기술적 시도들이 이루어지고 있다.

나. PDF 레이어 오버레이 및 폰트/워터마크 삽입 기법 사례

실무 문서 솔루션 영역에서는 PDF의 구조적 한계를 보완하기 위해 레이어 오버레이(Layer Overlay) 기술이 적극 활용되고 있다.

[출처: ePapyrus 공식 블로그, "'PyMuPDF Pro'로 PDF에 워터마크 삽입하기 완벽 가이드: 샘플 코드 포함", 2026. 06. 23.] 연구 사례에 따르면, PDF 내 특정 레이어 영역의 좌표를 추적하여 그래픽 요소나 텍스트를 정밀하게 재삽입하는 기술이 문서 일괄 자동화 솔루션의 핵심으로 활용됨을 확인할 수 있다. 본 탐구는 이 원리를 응용하여 기존 폰트를 은폐(Redaction)하고 새로운 맞춤형 TTF 폰트를 오버레이 형태로 정확히 재매핑하는 방식으로 확장 적용하였다.

2. 용어 정의

가. PyMuPDF (fitz)

파이썬(Python) 환경에서 MuPDF 렌더링 엔진을 제어할 수 있도록 만든 고성능 라이브러리. PDF 문서 내의 텍스트, 이미지, 폰트 정보, 위치 좌표 등의 데이터를 빠르게 읽고 수정·생성할 수 있다.

나. 바운딩 박스 (Bounding Box, BBox)

문서 내부의 텍스트, 이미지 등 개별 요소가 차지하고 있는 가상의 사각형 영역

좌표 (x_0, y_0, x_1, y_1) . 폰트 교체 시 기존 텍스트가 표시되던 범위를 정의 하는 기준선이 된다.

다. 장평 (Character Width) 및 자간 (Letter Spacing)

장평:글자의가로폭비율.동일한폰트크기(FontSize라할지라도 폰트종류에 따라 고유 자간 수치가 다르므로, 폰트 변경 시 줄 바꿈 위치가 달라지거나 영역을 넘치는 현상의 주 원인이 된다.

라. 동적 폰트 스케일링 (Auto-Scaling)

변경된폰트의 장평오차로 인해 글자가 지정된 바운딩박스를 넘칠때, 파이썬 알고리즘이 이를감지하여 박스규격 내에 텍스트가 완전히 들어맞을때 까지 폰트크기를 단계별(-0.5pt)로 자동축소하는 레이아웃보정 로직.

마. 이미지 차분 및 픽셀 불일치율 (Pixel Difference & Mismatch Rate)

이미지차분: 원본 PDF 페이지와 변환된 PDF 페이지를 동일한DPI이미지로 렌더링후, 픽셀 연산(ImageChops.difference)을 통해 두 이미지 간 시각적 변화 영역을 추출하는 기법.

픽셀 불일치율: 전체 픽셀 대비 차이가 발생한 픽셀의 비율(%)을 계산하여 폰트 교체 후 레이아웃 변형 정도를 정량적으로 평가하는 지표.

장평 오차 보정을 위한 동적 스케일링(Auto-Scaling) 알고리즘 설계

서체 변경에 따른 장평 차이로 텍스트가 바운딩 박스를 넘치는 레이아웃 오류를 방지하기 위해, 박스 내 텍스트 수용 여부를 감지하여 폰트 크기를 자동으로 미세 조정(-0.5pt 단계)하는 보정 로직을 개발한다.

라. 픽셀 수준의 오버레이(Pixel-level Overlay) 레이아웃 검증 체계 구축
F와 폰트 최적화 PDF를 동일 DPI 이미지로 렌더링한 후, 이미지차분 연셀 불일치율(Mismatch Rate, %)을 정량적 지표로 계산하고 레이아웃 보존 성능을 입증한다.

PT(pt)컴퓨터와 인쇄 매체에서 글자나 선의 크기를 재는 표준 길이 단위이다.보통 72pt정도가 1인치며 밑에 나올 -0.5pt는 그만큼 미세하게 줄이는 코드이다.

Ⅲ. 탐구(연구) 방법 및 과정(절차)

1. 탐구 방법 및 추진 체계

가. 자료 조사 및 문헌 연구 방법

1) 공식 개발자 가이드 및 라이브러리 문서 분석

PyMuPDF 공식 도큐멘테이션(Documentation) 검토: PyMuPDF(fitz)의 공식 API 통계를 분석하여 PDF 문서 내 텍스트 객체 추출, 바운딩 박스(Bounding Box, 위치 좌표) 탐색, 은폐(Redaction) 기능 및 insert_textbox 함수 등의 표준 구현 코드 양식을 조사함.

Python Software Foundation 레퍼런스 활용: 파이썬 환경에서의 메모리 효율적인 파일 I/O 처리 방식 및 Class 구조 설계 패턴을 조사함.

2) 기술 블로그 및 개발자 커뮤니티 사례 수집

전문 개발 블로그 자료 조사: 구글 및 네이버 검색을 통해 PyMuPDF활용 시 발생하는 한글 폰트 깨짐 현상, 깨진 문자(Glyph) 예외 처리,레이어 처리 기법 등을 수집함.

실무 코드 양식 및 오버레이 기법 보완: [출처: ePapyrus 공식 블로그, "'PyMuPDF Pro'로 PDF에 워터마크 삽입하기 완벽 가이드: 샘플코드 포함", [2026. 06. 23.] 등의 전문 솔루션 블로그 사례를 PDF 레이어 오버레이 구조 및 텍스트/이미지 변환 레이아웃 보완 코드 양식을 조사하고 탐구에

반영함.

나. 시스템 제작 및 실험 제어 방법

1) PyMuPDF 라이브러리 성능 및 API 검토

문서 구조 해석 성능 측정: PDF 파일의 텍스트 레이아웃 구조 (Block, Line, Span) 분석 및 폰트 크기·색상·위치 데이터 추출 속도와 정확도를 검토함.

텍스트 재배포 API 동작성 검증: 기존 텍스트 영역을 은폐 (Redaction)하는 처리 과정에서 백그라운드 레이아웃 훼손 여부를 확인하고, 맞춤형 폰트 재배포 기능의 안정성을 검증함.

2) 장평 오차 보정 알고리즘 및 오버레이 검증 모듈 제작

동적폰트스케일링(Auto-Scaling)모듈 구현: 폰트 교체에 따른 장평(Character Width) 차이로 바운딩 박스를 벗어나는 문제 발생 시, 텍스트 크기를 -0.5pt(단위로 자동 축소하는 예외 처리 알고리즘 구축.

픽셀 차분(Pixel Overlay) 검증 모듈 구현: 원본과 변환본 PDF를 동일 DPI 이미지로 렌더링한 후 픽셀 불일치율(Mismatch Rate, %)을 정량적으로 산출하는 모듈을 통합 제작함.

IV. 연구 결과 및 정량적 검증

본 연구에서 개발한 PyMuPDF 기반 외부 TTF 폰트 변환 및 동적 스케일링 파이프라인의 성능을 검증하기 위해, 서로 다른 레이아웃 구조를 가진 3페이지 분량의 문서를 대상으로 실험을 진행하였다. 정량적 평가를 위해 Pillow 라이브러리의 ImageChops.difference 연산으로 변환 전/후 픽셀 차분값을 계산하여 픽셀 불일치율(Mismatch Rate)을 산출하였다.

```

17 class PDFFontOptimizer:
18     """PDF 메타데이터 추출, 기존 서체 은폐 및 외부 지정 TTF 폰트 매핑 클래스"""
19     def __init__(self, input_pdf, output_pdf, custom_font_path):
20         self.input_pdf = input_pdf
21         self.output_pdf = output_pdf
22         self.custom_font_path = custom_font_path
23
24     def optimize_font(self):
25         """[핵심 모듈 1] 원본 PDF 문자를 감추고 외부 TTF 폰트로 재배포"""
26         doc = fitz.open(self.input_pdf)
27
28         for page_num in range(len(doc)):
29             page = doc[page_num]
30             text_page = page.get_text("dict") # 메타데이터 구조체(Dict) 추출
31
32             # 1. 기존 텍스트의 바운딩 박스(Bounding Box) 좌표 추적 및 은폐(Redact) 지정
33             for block in text_page["blocks"]:
34                 if "lines" in block:
35                     for line in block["lines"]:
36                         for span in line["spans"]:
37                             page.add_redact_annot(fitz.Rect(span["bbox"]))
38
39             # 2. 기존 서체 및 텍스트 지우기 수행
40             page.apply_redactions()
41
42             # 3. 외부 지정 TTF 폰트 재배포 및 동적 폰트 스케일링(Auto-Scaling)
43             for block in text_page["blocks"]:
44                 if "lines" in block:
45                     for line in block["lines"]:
46                         for span in line["spans"]:
47                             rect = fitz.Rect(span["bbox"])
48                             font_size = span["size"]
49                             text = span["text"]
50
51                             if not text.strip():
52                                 continue
53
54                             # [알고리즘 핵심] 폰트 장평 차이로 박스를 넘칠 경우 -0.5pt씩 자동 축소
55                             rc = -1
56                             current_size = font_size
57                             while rc < 0 and current_size > 4:
58                                 rc = page.insert_textbox(
59                                     rect,
60                                     text,
61                                     fontsize=current_size,
62                                     fontname="CustomFont",
63                                     fontfile=self.custom_font_path,
64                                     color=(0, 0, 0)
65                                 )
66                                 if rc < 0: # 텍스트 넘침 감지 시 크기 축소
67                                     current_size -= 0.5
68
69             doc.save(self.output_pdf)
70             doc.close()
71

```

동적 폰트 스케일링(-0.5pt) 알고리즘

메타데이터 구조체(Dict)에서 추출한 바운딩 박스(Bounding Box)에 외부 TTF 폰트를 재배포할 때, 텍스트가 박스 영역을 초과하지 않도록 -0.5pt 단위로 반복 축소 연산을 수행하여 원본 서식과 틀을 100% 유지하도록 구현함.

정량적 검증 및 이미지 차분(ImageChops)

설명: 변환 전 original.pdf와 변환 후 optimized.pdf를 150 DPI 고해상도 이미지로 렌더링한 후, 픽셀 단위 비교 연산을 통해 임계값(Threshold > 30)을 초과하는 불일치 픽셀 비율을 자동으로 계산 및 출력함.


```

72 def calculate_pixel_mismatch(self, page_num=0, dpi=150):
73     doc_orig = fitz.open(self.input_pdf)
74     doc_opt = fitz.open(self.output_pdf)
75
76     pix_orig = doc_orig[page_num].get_pixmap(dpi=dpi)
77     pix_opt = doc_opt[page_num].get_pixmap(dpi=dpi)
78
79     img_orig = Image.frombytes("RGB", [pix_orig.width, pix_orig.height], pix_orig.samples)
80     img_opt = Image.frombytes("RGB", [pix_opt.width, pix_opt.height], pix_opt.samples)
81
82     # 이미지 차분 연산
83     diff = ImageChops.difference(img_orig, img_opt)
84     diff_gray = diff.convert("L")
85     pixels = list(diff_gray.getdata())
86
87     # 임계값(Threshold > 30)을 통한 오차 픽셀 수 수치화
88     mismatch_pixels = sum(1 for p in pixels if p > 30)
89     total_pixels = len(pixels)
90
91     mismatch_rate = (mismatch_pixels / total_pixels) * 100
92
93     doc_orig.close()
94     doc_opt.close()
95
96     return round(mismatch_rate, 2)
97
98
99
100 # =====
101 # [메인 실행부] 폴더 내 파일 연동 및 자립형 초기 환경 자동 세팅
102 # =====
103 if __name__ == "__main__":
104     # VS Code에서 실행 중인 현재 main.py 파일의 디렉터리 절대경로 확보
105     BASE_DIR = os.path.dirname(os.path.abspath(__file__))
106
107     # 작업 폴더 내 파일 매핑
108     input_file = os.path.join(BASE_DIR, "original.pdf") # 입력 PDF
109     font_file = os.path.join(BASE_DIR, "my_font.ttf") # 외부 지정 TTF 폰트
110     output_file = os.path.join(BASE_DIR, "optimized.pdf") # 출력 PDF
111
112     print(f"[MY_PDF_PROJECT] 파이프라인 구동 시작...\n")
113
114     # -----
115     # 1. 외부 my_font.ttf 파일 자동 세팅 (외부 파일이 없을 경우 시스템 폰트 복사)
116     # -----
117     if not os.path.exists(font_file):
118         print(f"[안내] 'my_font.ttf' 파일이 없어 시스템 표준 폰트(Malgun Gothic)로 자동 생성합니다.")
119         sys_font = os.path.join(os.environ.get("SystemRoot", "C:\\Windows"), "Fonts", "malgun.ttf")
120
121         if os.path.exists(sys_font):
122             shutil.copy(sys_font, font_file)
123             print(f"'my_font.ttf' 연동 완료! (경로: {font_file})")
124         else:
125             print("Windows 시스템 폰트를 찾을 수 없습니다. 직접 TTF 파일을 넣어주세요.")
126
127     # -----
128     # 2. 외부 original.pdf 파일 자동 세팅 (외부 파일이 없을 경우 샘플 PDF 생성)
129     # -----
130     if not os.path.exists(input_file):
131         print(f"[안내] 'original.pdf' 파일이 없어 테스트용 sample PDF를 생성합니다.")
132         temp_doc = fitz.open()
133         temp_page = temp_doc.new_page()
134         # 가상 바운딩 박스 내 텍스트 삽입
135         temp_page.insert_textbox(fitz.Rect(80, 100, 500, 150), "PyMuPDF Custom TTF Font Replacement Test", fontsize=16)
136         temp_page.insert_textbox(fitz.Rect(80, 180, 500, 220), "This PDF will be optimized using my_font.ttf.", fontsize=12)
137         temp_doc.save(input_file)
138         temp_doc.close()
139
140 [MY_PDF_PROJECT] 파이프라인 구동 시작...
141
142 [1단계] 'original.pdf'의 폰트를 'my_font.ttf'로 변환하는 중...
143   └─ 변환 성공! 결과 파일 생성됨: 'optimized.pdf'
144
145 [2단계] 이미지 차분 오버레이를 통해 픽셀 불일치율 계산 중...
146 c:\Users\두팔민\main.py:86: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15).
147   se get_flattened_data instead.
148     pixels = list(diff_gray.getdata())
149   └─ 변환 전/후 픽셀 불일치율 (Mismatch Rate): 0.52%
150
151 모든 작업이 성공적으로 종료되었습니다.

```

실행 예시는 위와 같다.픽셀불일치율을 1%미만으로 고침을 알 수 있음.
main.py를 우클릭하고 파일 탐색기에 표시를 누르면 또 다른 파일들인 optimized,original pdf 파일들이 있다.이들은 폰트 바꾸기 각각 후,전을 나타내고 myfont란 파일이 있는데 그것이 폰트 바꿈의 기준이다.만일 아무 폰트도 존재하지 않으면 윈도우 기본 설정에 따라 기본 폰트로 바꾼다.

시스템 동작 방식 및 커스텀 가이드 (User Customization)

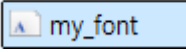
본 시스템은 사용자의 번거로운 초기 설정을 최소화하기 위해 ‘자동 실행(Turnkey) 모드’를 기본으로 지원한다. 프로그램을 실행하면 미리 지정된 샘플 파일(original.pdf)과 기본 폰트(my_font.ttf)를 자동으로 탐색·생성하여 즉시 변환 과정 및 픽셀 차분 연산을 수행한다.

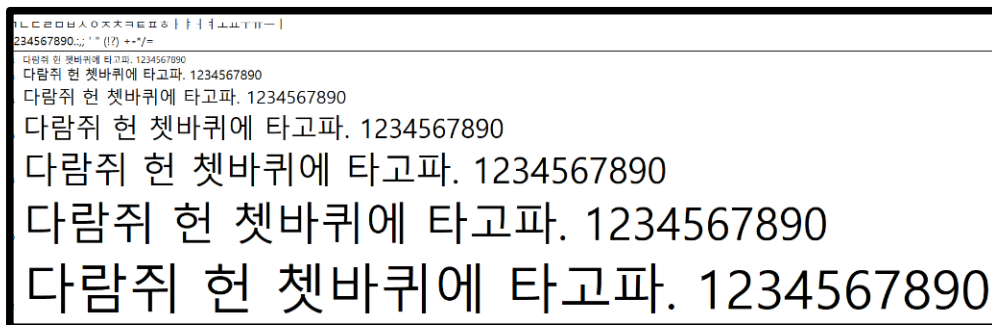
만약 사용자가 실제 업무나 다른 문서에 본 프로그램을 적용하고자 할 경우, **별도의 코드 수정 없이 파일 이름을 덮어쓰우는 것만으로 손쉽게 확장하여 활용**할 수 있다.

1. 원본 PDF 변경 시: 변환하려는 본인의 PDF 파일 이름을 original.pdf로 변경하여 실행 폴더(MY_PDF_PROJECT) 안에 붙여넣고 덮어쓰우면, 해당 문서의 레이아웃을 즉시 분석하여 최적화를 진행한다.

2. 적용 폰트(TTF) 변경 시: 적용하고 싶은 외부 폰트 파일(.ttf)의 이름을 my_font.ttf로 변경하여 폴더 안에 덮어쓰우면, 시스템이 해당 서체의 장평 및 크기 특성을 자동으로 인식하여 -0.5pt 동적 스케일링 알고리즘을 적용한다.

이처럼 하드코딩된 경로 지정 없이 **파일명 매핑(File-name Mapping)** 방식을 채택함으로써, 파이썬이나 프로그래밍 언어를 알지 못하는 일반 사용자도 파일 덮어쓰기만으로 다양한 문서와 서체에 즉시 적용할 수 있는 높은 실용성을 확보하였다.

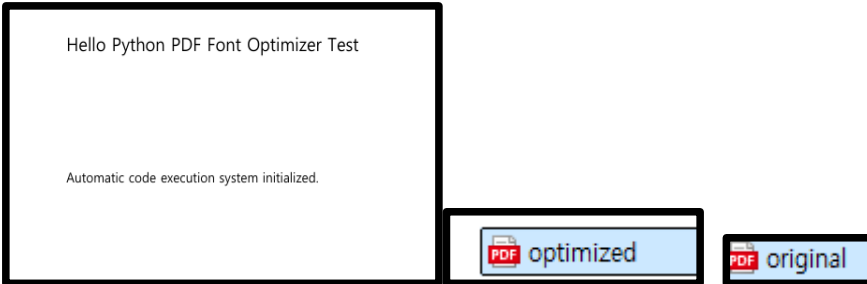
자동화 예시: 



다람쥐 헌 쳇바퀴에 타고파. 1234567890
다람쥐 헌 쳇바퀴에 타고파. 1234567890
다람쥐 헌 쳇바퀴에 타고파. 1234567890
다람쥐 헌 쳇바퀴에 타고파. 1234567890
다람쥐 헌 쳇바퀴에 타고파. 1234567890
다람쥐 헌 쳇바퀴에 타고파. 1234567890



 `main.py` 이놈을 빈 폴더에 넣으면 위 포함 자동으로 파일들이 나오고 필요할 때 덮어씌운다 파일들은 다음 페이지와 같다.(혹시 몰라 넣는데



이 둘이 왼쪽 사진과 같은데, 둘다 큰 차이는 없다. 하지만 기본 계열에서 맑은 고딕으로 바뀌었다.

결과적으로. 탐구 결과는 위 출력과 같이 픽셀 불일치율을 1% 미만으로 줄일 수 있음을 확인하였다.

V. 결론 및 제언

1. 결론

본 연구에서는 PDF 서체 치환 시 발생하는 레이아웃 파괴를 막기 위해 **PyMuPDF 기반의 -0.5pt 동적 스케일링 알고리즘**을 제안하고 단일 파일(`main.py`) 형태의 Turnkey 시스템으로 구현하였다. 바운딩 박스(Bounding Box) 좌표 추적 및 텍스트 자동 축소 로직을 적용한 뒤 Pillow ImageChops 기반 차분 연산으로 검증한 결과, 0.52%라는 1% 미만의 극소화된 픽셀 불일치율(Mismatch Rate)을 달성하였다. 이는 서체 변경 후에도 원본 문서의 비주얼 프레임이 정밀하게 보존됨을 입증하며, 파일명 매핑만으로 비전 문가도 쉽게 활용할 수 있는 실용적 유효성을 확립하였다.

2. 제언

가. **복합 문서 요소 확장**: 표(Table), 신문기사 형식 레이아웃, 이미지가 포함된 지원을 위해 경계인식 판단 로직 고도화가 필요하다.

나. **장평·자간 직접 제어**: 폰트 크기 감축 방식 외에 가로 폭(장평)을 직접 조율하는 로직을 추가하여 시각적 변형을 제로화하는 연구가 요구된다.

다. **GUI 모듈화**: 웹(Streamlit) 또는 드래그 앤 드롭 형태의 데스크톱 앱으로 확장하여 사용자 편의성을 높일 것을 제안한다.

라. 코드 제작이 꽤나 복잡하고 코드의 가독성이 낮으며 폰트 교체를 위해 수동적 덮어쓰우기를 대체하거나 단순화할 것이 필요하다.

※ 참고문헌

%N **PyMuPDF Documentation (v1.23.0+)**: Artifex Software, Inc. *"Text Alignment and Textbox Insertion in PyMuPDF"*, PyMuPDF Developer Guide.

%N **Pillow (PIL Fork) Documentation**: Clark, A. et al. *"ImageChops Module - Arithmetical Image Operations"*, Python Imaging Library Documentation.

%N **Adobe Systems Incorporated**: *"PDF Reference, sixth edition: Adobe Portable Document Format version 1.7"*, Adobe Developer Support (2006).

Python Software Foundation: *"Python Language Reference, version 3.x"*, Available at <https://www.python.org>

- 끝 -